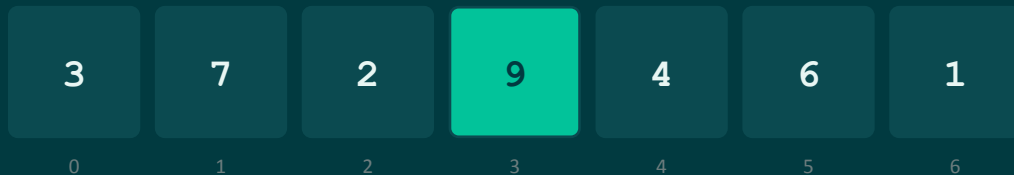


Three Techniques Every Problem Solver Needs

Prefix Sum • Binary Search • Two Pointers



What We'll Cover

01

Prefix Sum

Answer range-sum queries
in $O(1)$ after $O(n)$
preprocessing

02

Binary Search

Search a sorted space in
 $O(\log n)$ instead of $O(n)$

03

Two Pointers

Scan linear data with two
indices to cut $O(n^2)$ to $O(n)$

Prefix Sum — The Idea

THE PROBLEM

Given an array, answer many queries like “sum of elements from index L to R” — quickly.

NAIVE WAY

Loop from L to R every query $\rightarrow O(n)$ per query, $O(n \cdot q)$ total. Too slow for large inputs.

PREFIX SUM IDEA

Precompute $\text{prefix}[i]$ = sum of first i elements, once.
Then any range sum is just one subtraction:

KEY FORMULA

$$\text{sum}(L, R) = \text{prefix}[R] - \text{prefix}[L-1]$$

One subtraction $\rightarrow O(1)$ per query, after one $O(n)$ pass to build $\text{prefix}[]$.

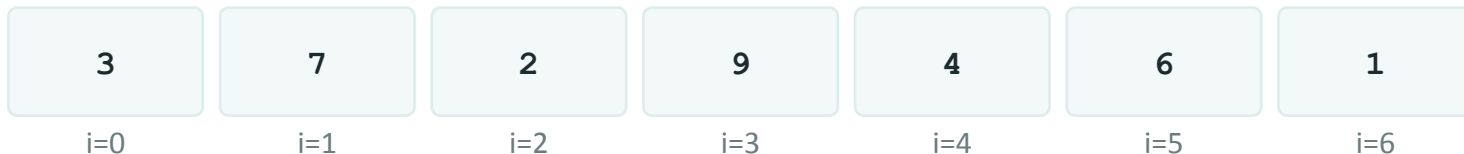
Think of it like a running total written on a whiteboard — to find the sum of a stretch, read two numbers off the board and subtract.

Worked Example, Step by Step

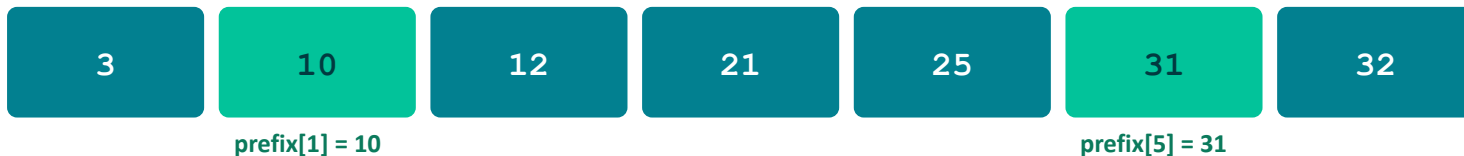
`arr = [3, 7, 2, 9, 4, 6, 1]`

Original array

Query: sum of index 2 to 5



Prefix array ($\text{prefix}[i] = \text{prefix}[i-1] + \text{arr}[i]$)



Query: sum from index 2 to 5 $\rightarrow \text{sum}(2,5) = \text{prefix}[5] - \text{prefix}[1]$
 $= 31 - 10 = 21$

Check by hand: $\text{arr}[2] + \text{arr}[3] + \text{arr}[4] + \text{arr}[5] = 2 + 9 + 4 + 6 = 21$ ✓ Matches — no loop needed at query time.

Pseudocode

BUILD THE PREFIX ARRAY — $O(N)$

```
prefix[0] = arr[0]
for i = 1 to n-1:
    prefix[i] = prefix[i-1] + arr[i]
```

ANSWER A RANGE QUERY — $O(1)$

```
function rangeSum(L, R):
    if L == 0: return prefix[R]
    return prefix[R] - prefix[L-1]
```

Why it works

- `prefix[i]` stores the running total of everything up to and including index `i`.
- `prefix[R]` holds “everything from 0 to R.” `prefix[L-1]` holds “everything from 0 to L-1.”
- Subtracting removes the part before `L`, leaving exactly the `L..R` stretch.
- Special case: if `L = 0`, there is nothing to subtract — just return `prefix[R]`.

C++ Implementation

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int n; cin >> n;
    vector<long long> arr(n), prefix(n);

    for (int i = 0; i < n; i++) cin >> arr[i];

    // Step 1: build prefix array
    prefix[0] = arr[0];
    for (int i = 1; i < n; i++)
        prefix[i] = prefix[i - 1] + arr[i];

    // Step 2: answer queries in O(1)
    int q; cin >> q;
    while (q--) {
        int L, R; cin >> L >> R;
        long long ans = (L == 0) ? prefix[R] : prefix[R] - prefix[L - 1];
        cout << ans << "\n";
    }
}
```

Binary Search — The Idea

THE PROBLEM

Find a target value in a sorted array as fast as possible.

NAIVE WAY

Check every element one by one $\rightarrow O(n)$. Fine for small arrays, slow for 10^6+ elements.

BINARY SEARCH IDEA

Because the array is sorted, check the middle element. It tells you which half the target must be in — throw the other half away. Repeat.

KEY IDEA

Each check halves the search space.

$n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots \rightarrow 1$ $\sim$
 $\log_2(n)$ steps total

Like looking up a word in a printed dictionary: you don't read page by page — you open the middle, then jump left or right depending on the letter.

Worked Example, Step by Step

Sorted arr = [2, 4, 7, 10, 15, 19, 24, 31]

Find target = 19

Step 1: lo=0, hi=7, mid=3 → arr[3]=10 < 19 → search right half



Step 2: lo=4, hi=7, mid=5 → arr[5]=19 == 19 → found!



RESULT

Target 19 found at index 5 in just 2 comparisons – a linear scan would need up to 6.

Each step compares arr[mid] to target: equal → done. Smaller → target is to the right, move lo = mid+1. Larger → target is to the left, move hi = mid-1.

Pseudocode

```
function binarySearch(arr, target, lo, hi):  
    if lo > hi:  
        return -1                //  
base case: not found  
  
    mid = lo + (hi - lo) / 2  
  
    if arr[mid] == target:  
        return mid                //  
found  
    else if arr[mid] < target:  
        return binarySearch(arr, target,  
mid + 1, hi)    // go right  
    else:  
        return binarySearch(arr, target,  
lo, mid - 1)    // go left
```

Loop invariant

- The array must be sorted first — binary search relies on order.
- lo..hi is always the region that could still contain the target.
- $mid = lo + (hi - lo) / 2$ avoids integer overflow vs $(lo + hi) / 2$.
- Each iteration shrinks lo..hi by half, so the loop runs $O(\log n)$ times.

C++ Implementation

```
#include<bits/stdc++.h>
using namespace std;

int binarySearch(vector<int>& arr, int target, int lo, int hi) {
    if (lo > hi) return -1;           // base case: not found

    int mid = lo + (hi - lo) / 2;

    if (arr[mid] == target) return mid;           // found
    else if (arr[mid] < target) return binarySearch(arr, target, mid + 1, hi); // go
right
    else return binarySearch(arr, target, lo, mid - 1);           // go left
}

int main() {
    vector<int> arr = {2, 4, 7, 10, 15, 19, 24, 31};
    int target = 19;
    cout << binarySearch(arr, target, 0, (int)arr.size() - 1) << "\n"; // prints 5
}
```

Two Pointers — The Idea

THE PROBLEM

In a sorted array, find two numbers that add up to a target sum.

NAIVE WAY

Try every pair with two nested loops $\rightarrow O(n^2)$. Slow once n grows large.

TWO POINTERS IDEA

Place one pointer at the start (left) and one at the end (right). Move them toward each other based on how the current sum compares to the target.

KEY RULE

`sum too small  $\rightarrow$  move left`

`pointer right`

`sum too big  $\rightarrow$  move right`

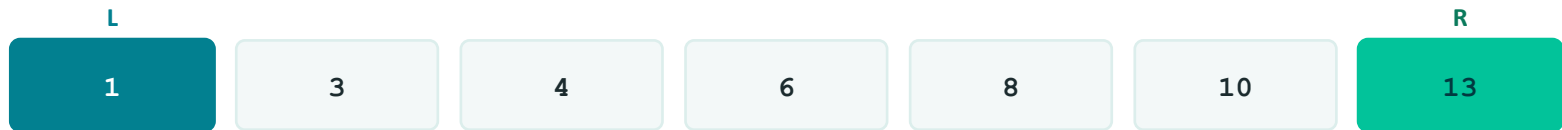
`pointer left`

Like two people walking toward each other from opposite ends of a sorted line of numbered rooms — each step narrows the search, so every element is visited once.

Worked Example, Step by Step

Sorted arr = [1, 3, 4, 6, 8, 10, 13]

Target sum = 14



1 + 13 = 14 = target → found pair!

(If the first check hadn't matched, we'd continue: $sum < target \rightarrow L++$, $sum > target \rightarrow R--$)

TRACE

`L=0 (val=1), R=6 (val=13) → sum = 14 → equals target → return (0, 6)`

If target had been 17 instead:

`L=0(1), R=6(13) → sum=14 < 17 → move L right → L=1(3), R=6(13) → sum=16 < 17 →`

`L=2(4), R=6(13) → sum=17 → found (2, 6)`

Pseudocode

```
function twoSumSorted(arr, target):  
    L = 0  
    R = length(arr) - 1  
  
    while L < R:  
        sum = arr[L] + arr[R]  
  
        if sum == target:  
            return (L, R)           // found  
        else if sum < target:  
            L = L + 1                // need  
bigger sum  
        else:  
            R = R - 1                // need  
smaller sum  
  
    return NOT_FOUND
```

Why it's $O(n)$

- Requires the array to be sorted first.
- L only moves right, R only moves left — they can move at most n steps combined.
- So the while loop runs at most n times total, not n^2 .
- Same pattern reused for: removing duplicates, merging sorted arrays, palindrome checks.

C++ Implementation

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    vector<int> arr = {1, 3, 4, 6, 8, 10, 13};
    int target = 14;

    int L = 0, R = (int)arr.size() - 1;
    bool found = false;

    while (L < R) {
        int sum = arr[L] + arr[R];

        if (sum == target) {
            cout << "Found pair: " << arr[L] << " + " << arr[R] << "\n";
            found = true;
            break;
        } else if (sum < target) L++;    // need a bigger sum
        else R--;                      // need a smaller sum
    }
    if (!found) cout << "No pair found\n";
}
```

Choosing the Right Technique

Technique	Best for	Requires	Complexity
Prefix Sum	Many range-sum queries	Nothing special	$O(n)$ build, $O(1)$ /query
Binary Search	Search / find boundary	Sorted data	$O(\log n)$
Two Pointers	Pair / subarray scans	Sorted data (usually)	$O(n)$

Rule of thumb: sort first, then reach for binary search (single target) or two pointers (pair/window); precompute prefix sums the moment you see “range sum” repeated many times.

PRACTICE

Try These Next

Prefix Sum

Equilibrium index of an array (index where left sum = right sum)

Binary Search

Find the first occurrence of a target in a sorted array with duplicates

Two Pointers

Given an array, find if any triplet sums to zero